

머신러닝 팀 프로젝트 · 취업 포트폴리오

LoL 경기는 언제 승부가 결정되는가

10분 시점의 팀 지표 13개로 승패를 판정하고 왜까지 설명하는 서비스를 만들었습니다.
로지스틱 회귀 하나로 — 정확도 0.7366, 찍기 대비 +23.8%p.

역할 — 데이터 분석 · 모델링 · 검증 체계 · 웹 서비스 구현 (4인 팀, 이제민)

GitHub github.com/wpalswpa/project2608

Live p4.sumzip.com

Stack Python · scikit-learn · Flask · MariaDB · Riot API

30초 요약 — 숫자 네 개

판정 정확도

0.7366

분할 10회 반복 평균

찍기(0.501) 대비 +23.8%p

접전 구간 정확도

0.615

골드차 1천 미만(34%)

— 정보의 한계를 공개

승리요인

1위 골드

골드 > 경험치 > 드래곤

두 방법(계수·permutation) 일치

자동 검증

21항목

파리티·수치 대조·회귀·계약

배포 전 전부 통과해야 함

이 프로젝트가 보여주는 것 세 가지

- ① **결론까지의 검증** — 시험지 1,976판을 봉인하고, 분할을 10번 바꿔도 결론이 유지되는지 확인했습니다.
- ② **이상 신호를 파고드는 힘** — "킬을 하면 진다"는 계수를 보고 멈추지 않고, 다중공선성을 진단해 원인을 증명했습니다.
- ③ **한계를 먼저 말하는 문화** — 접전은 못 맞힌다는 사실을 숨기지 않고 화면에 경고로 띄웁니다.

질문 두 개, 그리고 설계 원칙 하나

Q1. 10분 정보로 승패를 얼마나 맞히나?

경기의 1/3은 10분에 이미 기울어 있다 — 그 경계를 숫자로 긋는다

Q2. 무엇이 승패를 가르나?

"왜"를 반드시 낸다 — 이 요구가 모델·피쳐 선택을 전부 결정했다

설계 원칙 — 분할을 별도 단계로

CRISP-DM 을 따르되 한 곳을 바꿨습니다. 분할은 잘못돼도 오류가 나지 않으면서 이후 점수를 전부 거짓으로 만드는 유일한 단계라서, 어떤 전처리보다 먼저 시험지 1,976판을 봉인하고 겹침·비율·분포를 기계로 진단했습니다.

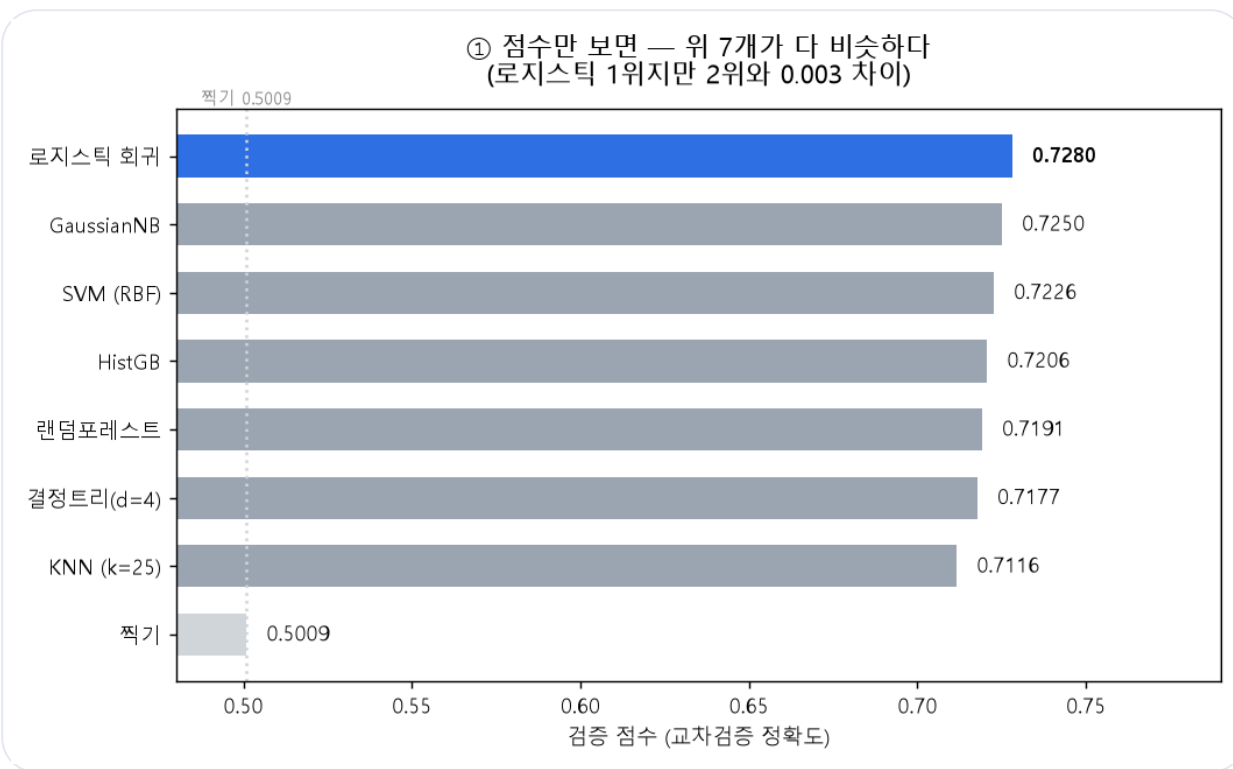
데이터 — Kaggle 다이아 랭크 9,879판 (결측 0 · 중복 0 · 승패 50.1 : 49.9)



다섯 단계 중 분할만 별도 단계로 승격 — 이 프로젝트의 변형입니다.

후보 8종을 같은 조건에서 — 점수로는 못 고른다

1위와 2위가 0.003 차이



점수 대신 두 가지로 골랐습니다

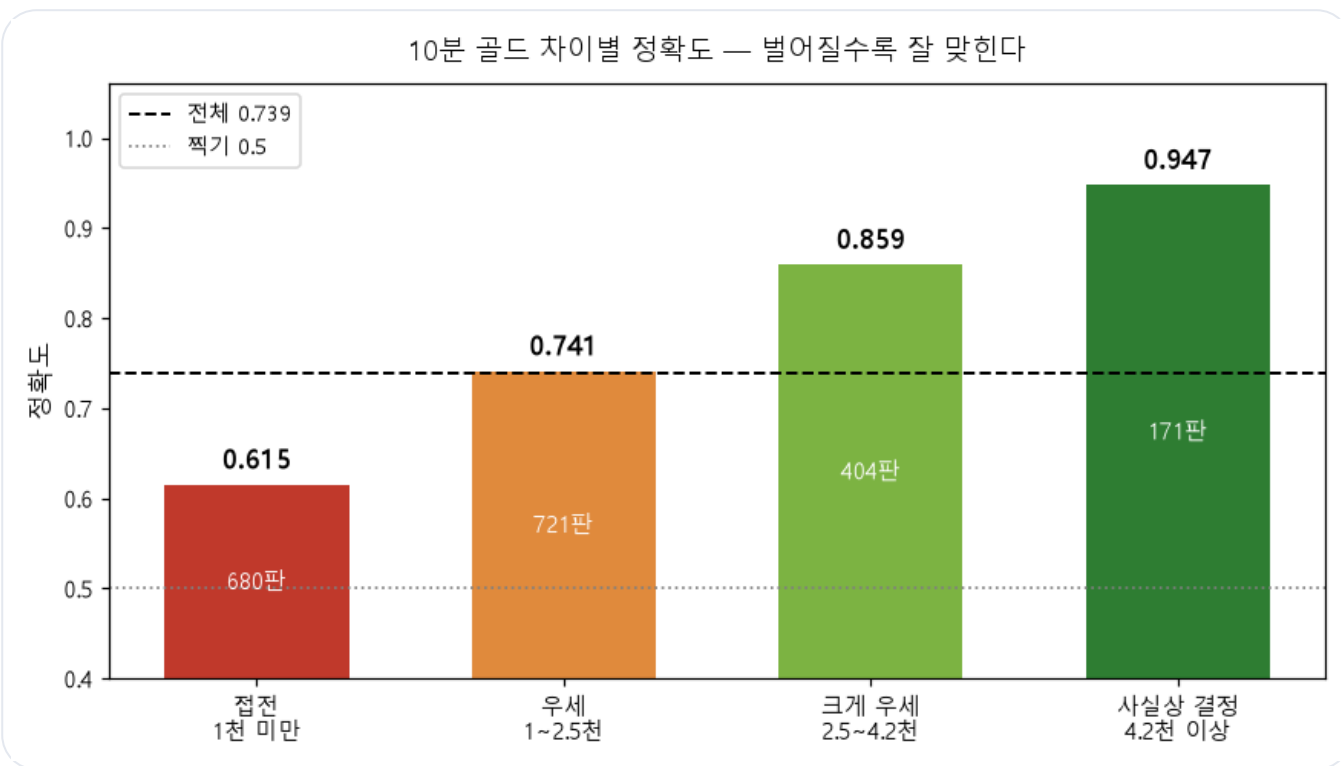
① 외웠는가 — 연습·검증 점수 격차. 복잡한 모델일수록 크게 벌어졌습니다.

② 왜를 말할 수 있는가 — 랜덤포레스트는 중요한 지표는 말해도 유·불리 방향(+/-)을 못 말합니다.

로지스틱 회귀만 세 기준을 전부 통과 — 성능이 복잡도의 역순이었습니다. 10분 정보량이 적어, 복잡한 모델은 배울 게 없어 외우기만 했습니다.

구간별로 쪼개 보면 — 잘 맞는 곳과 못 맞는 곳

홀드아웃 1,976판 · AUC 0.8150 · Brier 0.1756



같은 모델인데 구간마다 33%p 차이

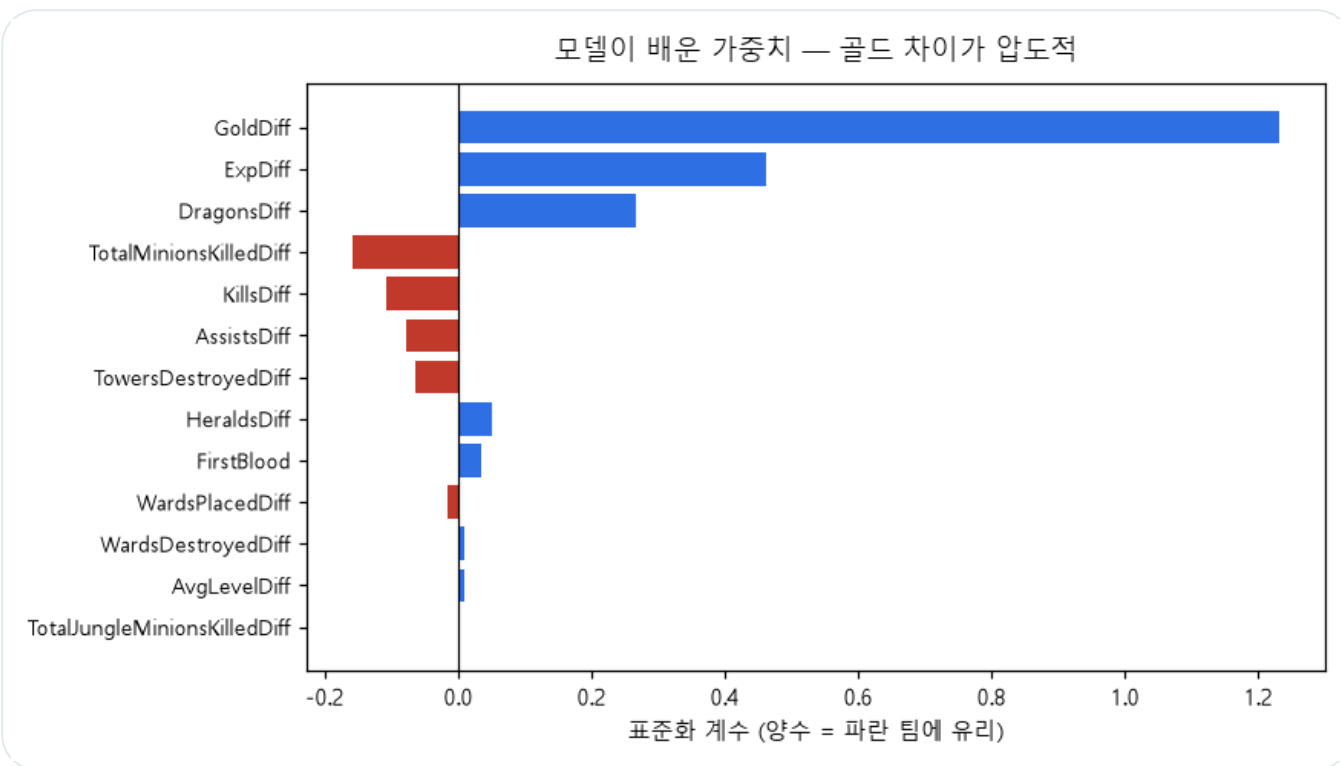
골드차 4.2천 이상은 **94.7%** 맞하지만, 1천 미만 접전은 **61.5%** 입니다.

이건 모델이 아니라 정보의 한계입니다.

접전 구간의 실제 승률이 49.5% — 10분 데이터 자체에 승패의 단서가 없습니다. 이 한계를 성적표로 공개하고 화면에 경고를 띄웁니다.

킬을 많이 한 팀이 불리하다? — 이상한 계수 4개

여기서 멈추지 않은 것이 이 프로젝트의 핵심



모델이 배운 가중치 13개를 펼치니

골드가 +1.231 로 압도적 1위 — 여기까진 예상대로.

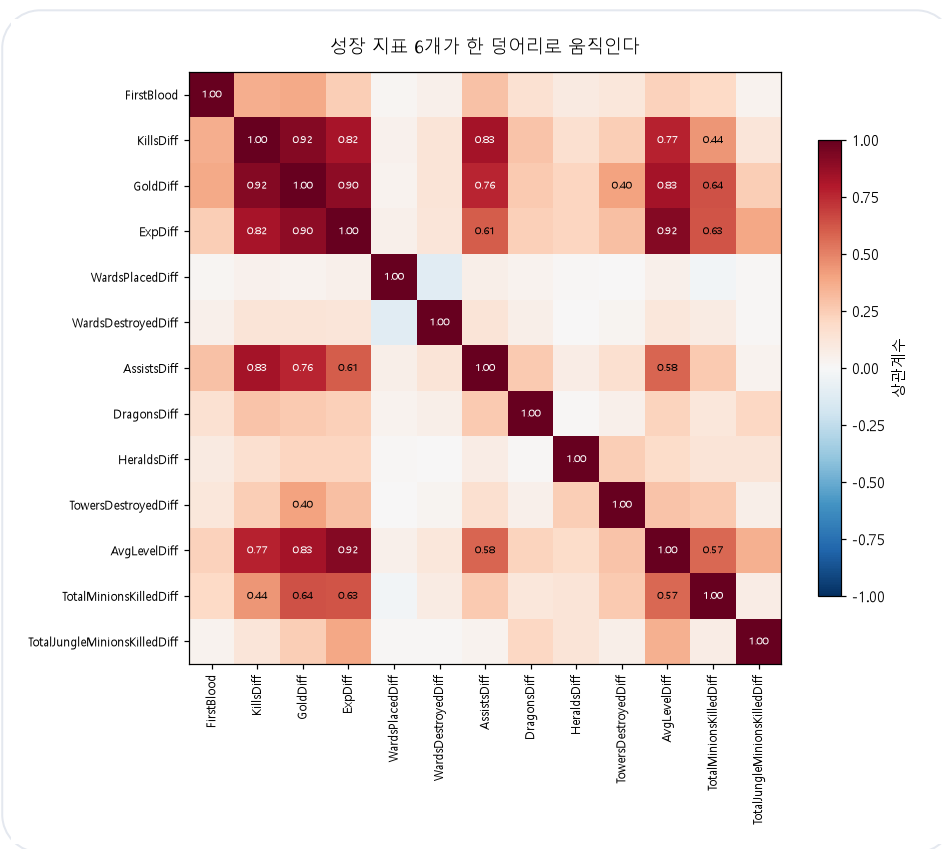
그런데 **킬·미니언·어시스트·타워가 음수**였습니다.

킬은 단독으로 판별력 3위입니다.

관련 있는 지표가 왜 음수인가 — 이 수수께끼를 다음 장에서 풀니다.

원인 — 지표들이 한 몸이었다

킬 ↔ 골드 상관 +0.92



킬도 미니언도 타워도 전부 돈이 됩니다

성장 지표 6개가 상관 0.9로 얹혀, 골드를 이미 본 모델에게 킬은 새로 알려줄 것이 없습니다. 남은 몫의 부호가 음수로 나온 것 — 다중공선성입니다.

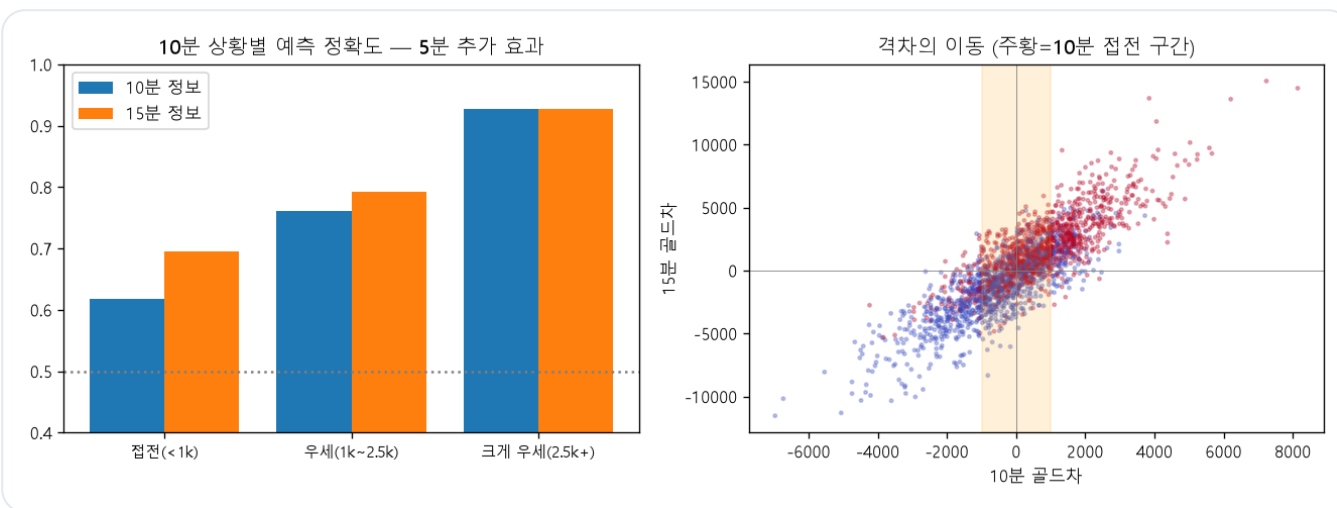
검증 — 골드를 빼고 다시 학습

음수로 눌러 있던 5개가 전부 양수로 살아나고, 정확도는 **0.002**만 떨어졌습니다. 골드가 특별했던 게 아니라 다른 지표가 같은 정보를 들고 있던 것입니다.

서비스 코칭도 이 발견대로 — 계수를 그대로 쓰면 "CS를 먹지 마라"가 되므로, 따라오는 골드까지 함께 계산합니다.

10분 vs 15분 — 5분의 가치는 접전에만 있다

Oracle's Elixir 프로 경기 10,656판 · 조건 통제



전체 +5.25%p — 그런데 쪼개 보면

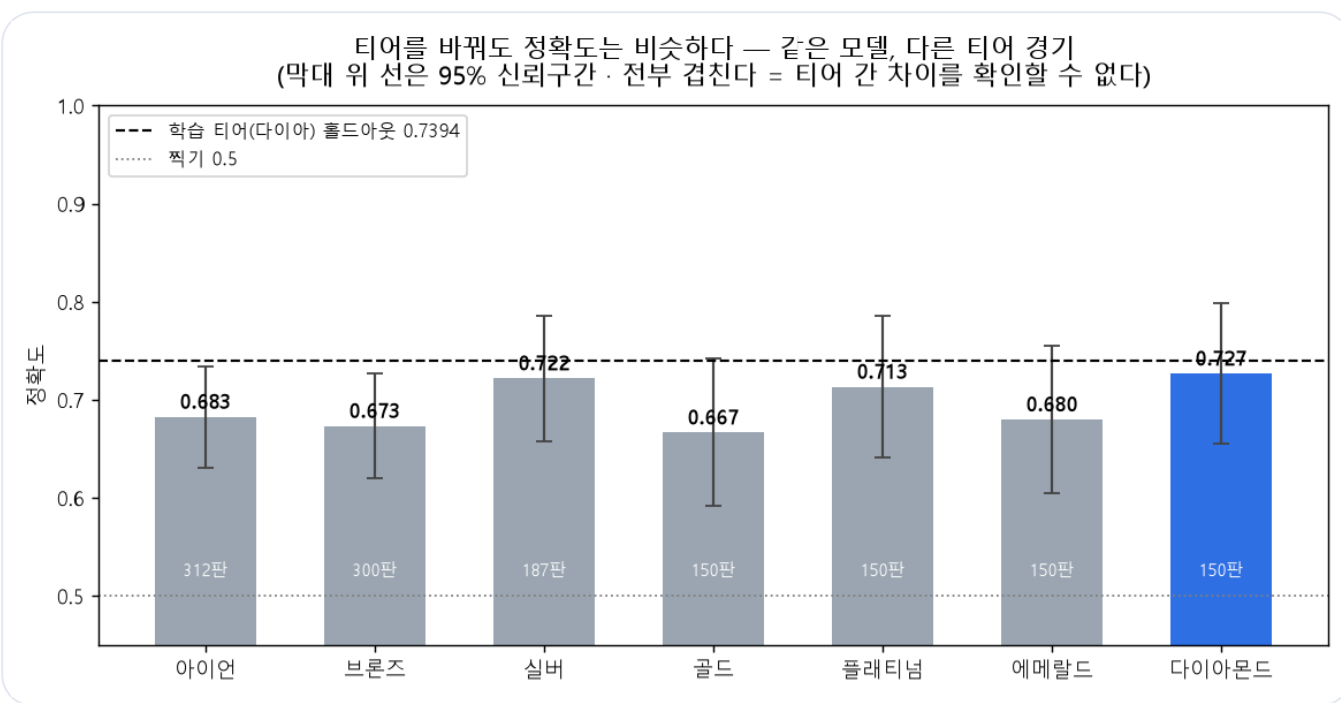
이미 벌어진 경기는 $\pm 0.0\%p$, 팽팽한 경기만 **+7.8%p** 오릅니다.

10분에 팽팽했던 판의 59%가 그 5분 사이에 한쪽으로 기울입니다.

결론 — 경기의 1할은 10분에 이미 끝나 있고, 팽팽한 절반은 그다음 5분에 갈립니다.

다른 티어에도 통하나 — 주장 대신 실측

2026년 아이언~다이아 1,399판 · 직접 수집



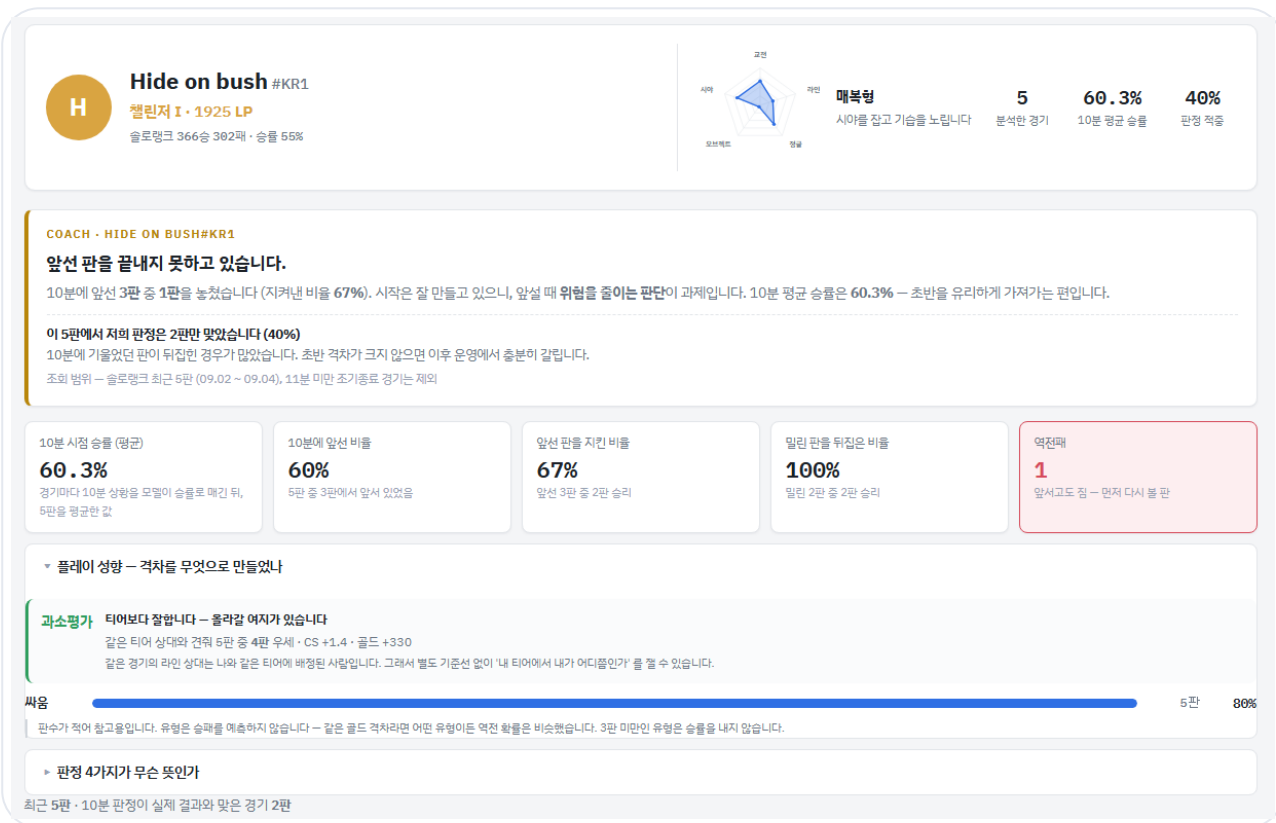
티어 차이는 확인되지 않았습니다

저티어 0.685 vs 고티어 0.707 · $z=0.83$ · $p=0.407$ — 신뢰구간이 전부 겹칩니다.

과장하지 않는 문장 — 티어당 150~312판이라 5%p 이내 차이는 검출력이 없습니다. 그래서 "차이가 없다"가 아니라 "큰 차이는 없다"까지만 적었습니다.

모델이 화면이 되기까지 — 판정 · 코칭 · 성향

p4.sumzip.com · 탭 4개 운영 중



판정 — 10분 확률 × 실제 결과

우세승 · 역전승 · 역전패 · 열세패. 같은 패배라도 처방이 다릅니다.

코칭 — "드래곤 하나 더였다면 +2.6%p"

관찰 기록임을 화면에 명시 — 인과 증명이 아닙니다.

성향 레이더 — 다섯 축 백분위

축끼리 상관 최대 0.15 가 되도록 데이터로 선정했습니다.

아키텍처

계산은 predict.py 한 곳 — 웹엔 sklearn 이 없습니다. 학습 산출물은 전처리 포함 Pipeline 2.2KB 하나.

스스로를 검증하는 프로젝트

배포 전 21항목 전부 통과

파리티

손계산 = 모델 = 라이브 화면

0.818255 가 소수점까지 일치
— 자동 테스트로 상시 대조

수치 대조

문서 24개 ↔ 근거 CSV

본문 속 모든 수치를 기계로 검증
— 현재 어긋남 0건

회귀

예측 50건 골든 정답지

모델·피쳐 변경 시
결과가 바뀌면 즉시 실패

계약

API 응답 ↔ 명세(serving.md)

필드·개수·타입까지 검사
— PUUID 유출 검사 포함

원칙 — 새 검증 규칙은 틀린 값을 넣어 "실제로 잡히는지" 반대 방향으로 확인합니다. 이 체계로 배포 전 회귀 세 건(화면 먹통 · 수치 불일치 · 좌표 밀림)을 잡았습니다.

이 프로젝트에서 배운 것

— 이상한 숫자는 버그가 아니라 단서다

킬 계수가 음수로 나왔을 때 모델을 의심하는 대신 데이터 구조(상관 0.9)를 진단했고, 그 발견이 페이지 2와 서비스 코칭 설계로 이어졌습니다.

— 한계는 숨기는 게 아니라 측정하는 것이다

접전 0.615, 티어 $p=0.407$, 2단계 라우팅 4종 열위 — 못 하는 것을 실측해 문서와 화면에 그대로 적었습니다. 신뢰는 정확도가 아니라 정직에서 나온다고 생각합니다.

— 계산은 한 곳에만 둔다

예측 로직을 `predict.py` 한 곳에 두고 화면·문서가 그 결과를 빌려 쓰게 했습니다. 화면 확률과 모델 확률이 어긋날 길 자체를 없앴습니다.

골드차 4천이면 끝난 경기, 1천 미만이면 이제 시작입니다

데이터로 확인한 이 한 문장이 프로젝트의 답입니다.

GitHub github.com/wpalswpa/project2608 Live p4.sumzip.com

README 에 전체 분석 과정 · docs/ 에 재현 절차 · final/ 에 산출물 6종

0.7366

판정 정확도 · 10회 평균

+23.8%p

찍기 대비 이득

p=0.407

티어 차이 — 확인되지 않음

21항목

배포 전 자동 검증